

Business Analysis — Terms and Tools

❖ Requirements

Requirement — a written statement of something the solution must do. The whole discipline rests on these being clear, testable and agreed. Its job is to remove ambiguity before money gets spent.

Functional requirement — what the system does. "The system sends a confirmation email after checkout."

Non-functional requirement — how well it does it. Speed, security, uptime, accessibility, how many users it handles at once. Projects usually get into trouble here, because these go unstated until something breaks.

Business requirement — the high-level goal in business terms. "Reduce order processing time by 40%." Answers *why the project exists*.

Stakeholder requirement — what a specific group needs. The warehouse team needs barcode scanning; finance needs an audit trail.

Solution requirement — the detailed level developers build from. Splits into functional and non-functional.

Transition requirement — temporary things needed only to get from old system to new: data migration, staff training, running both systems in parallel for a month. They disappear once the change is complete.

Elicitation — the work of drawing requirements out of people through interviews, workshops, observation and document review. People rarely state what they need accurately, so this is an active investigation rather than order-taking.

Validation vs verification — validation asks "are we building the right thing?" Verification asks "did we build it correctly?" Easy to confuse, and asked in nearly every certification exam.

Prioritisation — deciding what gets built first when everything is urgent to somebody.

Traceability — the ability to follow a requirement forward into design, code and tests, and backward to the business goal that justified it. Its job is to make sure nothing was quietly dropped and nothing was built without a reason.

Requirements Traceability Matrix (RTM) — the table that records those links.

Baseline — the agreed, frozen version of requirements at a point in time. Everything after is measured as change against it.

❖ People and governance

Stakeholder — anyone affected by the project or able to affect it. Includes people who will resist it.

Stakeholder analysis — mapping who they are, how much power they hold, how interested they are, and how to engage each one. Prevents the situation where a senior manager nobody consulted blocks the project two weeks before launch.

RACI matrix — a grid showing who is Responsible, Accountable, Consulted and Informed for each activity. Kills the "I thought you were doing that" problem.

Subject Matter Expert (SME) — the person who actually knows how the current process works in practice, which is usually different from how the manual says it works.

Product Owner — in Scrum, the single person who owns the backlog and decides priority.

Sponsor — the senior person funding the project and clearing obstacles. If a project has no active sponsor, it dies slowly.

Sign-off — formal written agreement that a deliverable is accepted. Its job is to stop the endless reopening of settled decisions.

❖ Documents

Business Case — the argument for doing the project at all: costs, benefits, risks, alternatives. Written before approval.

Project Charter — the short document that formally starts the project and names the sponsor, objectives and high-level scope.

BRD (Business Requirements Document) — the "why and what" for a business audience. No technical detail.

FRD / SRS — the detailed "how it behaves" for developers and testers.

User story — a requirement written from the user's point of view: *As a [role], I want [capability], so that [benefit]*. Keeps the reason attached to the request, which stops teams building features nobody wanted.

Epic — a large body of work too big for one sprint, later broken into stories.

Acceptance criteria — the specific conditions a story must satisfy to be accepted. Its job is to settle in advance what "finished" means.

Definition of Done — the team's standard checklist applied to every story: code reviewed, tested, documented, deployed. Different from acceptance criteria, which are per-story.

Use case — a step-by-step description of a user achieving a goal with the system, including alternative and failure paths.

Business rules — the policies the system must enforce, independent of any screen. "Refunds are only permitted within 30 days." These outlive the software and belong in their own catalogue.

❖ Analysis techniques

As-is analysis — documenting how the process works today, honestly, including the workarounds.

To-be analysis — designing how it should work.

Gap analysis — the difference between the two, and what's needed to close it. Drives the scope of the project.

Root cause analysis — tracing a problem past its symptoms to the real source, so you fix the cause rather than the complaint.

5 Whys — asking "why" repeatedly until you reach the root cause. Simple, fast, works in a meeting.

Fishbone (Ishikawa) diagram — a visual root cause tool sorting possible causes into categories like people, process, equipment, materials.

SWOT — strengths, weaknesses, opportunities, threats. Internal and external factors on one page. Used in strategy conversations.

PESTLE — political, economic, social, technological, legal, environmental. Scans the outside world for factors that could affect the project.

MoSCoW — prioritisation into Must, Should, Could and Won't have. The "Won't" column is the valuable one, because it records what was consciously excluded.

Kano model — sorts features into basics (expected, invisible when present, fatal when missing), performance (more is better) and delighters (unexpected pleasures). Helps explain why spending on basics never gets praised.

Cost-benefit analysis — comparing what it costs against what it returns.

ROI (Return on Investment) — the payback expressed as a percentage or period. The number executives ask for.

Feasibility study — can this actually be done given the budget, timeline, technology and people available.

Impact analysis — before making a change, working out everything it touches. Prevents the small fix that breaks four other systems.

Risk assessment — identifying what could go wrong, how likely, how bad, and what to do about it.

Benchmarking — comparing your process or numbers against competitors or industry standards.

Decision matrix — scoring options against weighted criteria so a choice can be defended with something other than opinion.

❖ **Modelling and diagrams**

Process mapping — drawing a process as a flowchart so everyone can see the steps, decisions and handoffs.

BPMN (Business Process Model and Notation) — the standard symbol set for process diagrams, so a diagram means the same thing to everyone who reads it.

Swimlane diagram — a process map with horizontal lanes per role or department. Its job is to expose handoffs, which is where delay and error concentrate.

UML (Unified Modeling Language) — a family of software modelling diagrams. The ones a BA touches most:

- **Use case diagram** — actors and the functions available to them
- **Activity diagram** — flow of actions, close to a flowchart
- **Sequence diagram** — the order of messages between components over time

Data Flow Diagram (DFD) — where data comes from, what transforms it, where it's stored, where it goes.

Context diagram — the whole system as a single box with everything it talks to around it. Good first slide in any workshop.

ERD (Entity Relationship Diagram) — how data entities relate. Customer has many orders; each order has many line items.

Data dictionary — a definition for every field: name, type, length, allowed values, meaning. Its job is to stop two departments meaning different things by "active customer".

Value stream map — a process map that also records time and waste at each step. From Lean.

Customer journey map — the process viewed as the customer experiences it, including their feelings and frustrations, not just the internal steps.

Wireframe / mockup / prototype — rough grey-box sketch, polished visual, then a clickable fake. Each stage lets you catch mistakes while they're still cheap to fix.

❖ Delivery approaches

SDLC (Software Development Life Cycle) — the overall stages a system passes through: requirements, design, build, test, deploy, maintain.

Waterfall — sequential phases, each finished and signed off before the next. Suits stable, well-understood requirements and regulated environments.

Agile — build in small increments, show it to users, learn, adjust. Trades upfront certainty for lower risk of building the wrong thing.

Scrum — the most common Agile framework: fixed-length sprints, defined roles, set ceremonies.

Kanban — continuous flow with work-in-progress limits instead of fixed sprints. Suits support and maintenance work.

Sprint — a fixed timebox, usually two weeks, ending in something potentially shippable.

Backlog — the ordered list of everything still to build.

Backlog refinement — the recurring session where items are clarified, estimated and reordered.

Story point — a relative size estimate, not hours. Its job is to allow estimating without pretending to a precision nobody has.

Velocity — how many points a team completes per sprint. Used for forecasting, misused as a performance target.

Burndown chart — remaining work plotted against time, showing whether the sprint is on track.

Stand-up, sprint review, retrospective — daily sync; demo to stakeholders at sprint end; team discussion on improving how they work.

MVP (Minimum Viable Product) — the smallest release that delivers real value and generates real learning.

Roadmap — the planned sequence of releases over months. Communicates direction without committing to dates for everything.

❖ Scope and change

Scope — the agreed boundary of what's included. Written as explicit in-scope and out-of-scope lists.

Scope creep — uncontrolled expansion of that boundary through accumulated small additions.

Change request — the formal mechanism for asking to alter something already baselined, so it gets assessed and costed instead of absorbed silently.

Change control — the process and approval body governing those requests.

Constraint — a fixed limitation you must work inside: budget, deadline, mandated technology, legislation.

Assumption — something taken as true without proof. Its job is to be written down, because unrecorded assumptions become failures nobody saw coming.

Dependency — something that must happen before your work can proceed, often owned by another team.

RAID log — one register holding Risks, Assumptions, Issues and Dependencies, reviewed regularly.

Issue log — problems that have already materialised and need resolving.

❖ Testing and rollout

Test case — a specific set of steps and expected results proving a requirement works.

UAT (User Acceptance Testing) — real business users confirming the system supports their actual work before go-live. The BA usually writes the scenarios and coordinates the sessions.

Regression testing — retesting existing functionality to confirm the new change didn't break it.

Defect / bug triage — reviewing reported defects and deciding severity, priority and who fixes what before release.

Go-live / cutover — the switch to the new system, and the detailed plan for that moment.

Rollback plan — how to revert if go-live fails. Nobody thinks about it until they need it.

Hypercare — the intensive support period straight after launch, when the team stands by to fix things fast.

Post-implementation review — an honest look back at whether the promised benefits actually materialised. Frequently skipped, which is why the same mistakes recur.

❖ Data and measurement

KPI (Key Performance Indicator) — the metric that shows whether an objective is being met. Choosing badly makes people optimise for the wrong behaviour.

SLA (Service Level Agreement) — a committed service standard, usually contractual, with consequences attached.

Data migration — moving data from the old system to the new one. Routinely underestimated.

Data mapping — specifying which field in the old system becomes which field in the new one, and how values are transformed.

ETL (Extract, Transform, Load) — the technical pattern for moving and reshaping data between systems.

Data quality — whether data is accurate, complete, consistent and current. Bad data undermines an otherwise good system.

Single source of truth — one authoritative place for a given piece of data, so departments stop arguing over whose spreadsheet is right.

Business and domain

Business process — a repeatable sequence of activities producing an outcome for a customer.

Process improvement / BPR — incremental refinement versus radical redesign from scratch.

Lean — a method focused on eliminating waste from processes.

Six Sigma — a data-driven method focused on reducing variation and defects. Often paired as Lean Six Sigma.

Capability — what the business is able to do, described independently of how it currently does it. Useful because capabilities stay stable while processes and systems change.

Legacy system — an older system still critical to operations, usually hard to change and poorly documented.

Integration / API / middleware — making systems talk to each other; the defined interface they talk through; the software sitting between them.

Build vs buy — whether to develop custom software or purchase an existing product and configure it.

RFP (Request for Proposal) — the formal document issued to vendors describing requirements and inviting bids. BAs often write the requirements section.

❖ Frameworks and certifications

BABOK — the Business Analysis Body of Knowledge, published by **IIBA**. The reference text for the profession.

ECBA / CCBA / CBAP — IIBA certifications for entry, intermediate and senior levels.

PMI-PBA — the Project Management Institute's business analysis certification.

IIBA-AAC — Agile Analysis Certification, for BAs working in Agile settings.

TOGAF — an enterprise architecture framework, relevant to BAs working at the architecture level.

ITIL — the service management framework, relevant where the work touches IT operations and support.

Tool Vocabulary Explained

❖ Jira

1. **Issue** — the generic name for any item in Jira, whether it's a story, a bug or a task. Everything in Jira is an issue.
2. **Issue type** — the category: **Epic** (a large chunk of work spanning sprints), **Story** (a user-facing requirement), **Task** (work with no direct user benefit, like setting up a server), **Sub-task** (a piece of a story), **Bug** (something broken).
3. **Project** — a container holding related issues, usually one per product or team.

4. **Board** — the visual wall of cards. A **Scrum board** works in fixed sprints; a **Kanban board** flows continuously with no sprint boundaries.
5. **Backlog** — the ordered list of issues not yet in a sprint.
6. **Sprint** — a fixed timebox, usually two weeks, with a committed set of issues.
7. **Workflow** — the defined path an issue travels, e.g. To Do → In Progress → In Review → Done. **Status** is where it sits now; **transition** is the move between statuses. Workflows can enforce rules, like blocking a move to Done until testing is complete.
8. **Swimlane** — horizontal rows grouping cards on a board, often by assignee, epic or priority.
9. **Story points** — a relative size estimate rather than hours. A 5 is roughly five times a 1.
10. **Priority vs Severity** — priority is how soon it must be fixed; severity is how badly it breaks things. A cosmetic typo on the homepage can be low severity but high priority. Interviewers like this distinction.
11. **Assignee vs Reporter** — who's doing the work, versus who raised it.
12. **Epic link** — the field tying a story to its parent epic, which is how you roll up progress.
13. **Linked issues** — relationships between issues: *blocks, is blocked by, relates to, duplicates*. Blocking links are how dependencies get tracked.
14. **Component** — a subsection of the product, like Payments or Search, used for filtering and routing.
15. **Label** — a free-text tag for grouping issues across projects. Easy to create and easy to make a mess with.
16. **Fix version / Release** — which release an issue is scheduled to ship in.
17. **Sprint report** — what was committed vs completed at sprint end.
Burndown — remaining work over time. **Velocity chart** — points completed per sprint, used for forecasting. **Cumulative flow diagram** — work in each status over time; widening bands reveal bottlenecks.
18. **JQL (Jira Query Language)** — the query syntax for finding issues, e.g. `project = ABC AND status != Done AND assignee = currentUser()`. This genuinely marks you out, because most people only click the filter dropdowns.
19. **Filter / Saved filter** — a stored JQL query you can reuse and share.
20. **Dashboard and Gadget** — a custom page of charts and lists. Gadgets are the individual panels.

21. **Custom field** — an extra field added to capture something Jira doesn't track by default, such as business unit or benefit value.
22. **Bulk edit** — changing many issues at once. Saves hours when reorganising a backlog.
23. **Automation rule** — if-this-then-that logic, e.g. when all sub-tasks close, move the parent to Done.

❖ Confluence

1. **Space** — a top-level container, usually one per team or project.
2. **Page / Child page / Page tree** — pages nest inside each other, forming the navigable hierarchy on the left.
3. **Template / Blueprint** — a pre-built page structure. Blueprints go further, adding indexes and logic. Using the requirements blueprint means every requirement page looks the same, which makes reviewing far faster.
4. **Macro** — an embedded widget adding function to a page. The **table of contents macro** auto-builds navigation; the **Jira issues macro** pulls live Jira data into the document so status never goes stale.
5. **Inline comment vs Page comment** — a comment attached to highlighted text, versus a general one at the bottom. Inline comments are how requirements reviews actually happen.
6. **Page history / Version comparison** — every edit is stored, and you can diff two versions. Essential when someone asks what changed since sign-off.
7. **Restrictions** — limiting who can view or edit a page, separate from space-wide permissions.
8. **Watch** — subscribing to notifications when a page changes.

❖ Excel

1. **Pivot table** — summarises a large table by dragging fields into rows, columns and values. Turns 50,000 rows into an answer in about thirty seconds. The single most valuable Excel skill.
2. **Slicer** — clickable filter buttons attached to a pivot table, making it feel like a small dashboard.
3. **VLOOKUP / XLOOKUP / INDEX-MATCH** — ways of pulling a value from another table by matching a key. XLOOKUP is the modern one and handles cases the older functions can't.

4. **IF / IFS** — conditional logic. **SUMIF / COUNTIF** and their plural forms sum or count only rows meeting criteria.
5. **Absolute vs relative reference** — \$A\$1 stays fixed when copied; A1 shifts. Nearly every broken spreadsheet formula traces back to this.
6. **Named range** — giving a range a readable name so formulas say TaxRate instead of \$B\$14.
7. **Data validation** — restricting what can be typed into a cell, usually as a dropdown list. Keeps data clean at entry.
8. **Conditional formatting** — colouring cells by rule, e.g. red where variance exceeds 10%. Makes exceptions jump out.
9. **Text to Columns / Remove duplicates** — quick cleanup tools for splitting messy fields and deduplicating lists.
10. **Power Query** — the proper tool for importing, cleaning and merging data from multiple sources, with the steps recorded so they replay on refresh. It replaces the manual copy-paste-clean cycle most people repeat every month.
11. **Power Pivot / Data model** — relating multiple tables inside Excel, so you can pivot across them without merging first.
12. **Goal Seek / Scenario Manager** — what-if analysis. Goal Seek works backwards from a target; Scenario Manager compares saved sets of assumptions.
13. **Macro / VBA** — recorded or written automation for repetitive tasks.
14. **Table (structured reference)** — converting a range into a formal table, so formulas use column names and ranges expand automatically as rows are added.

❖ SQL

1. **SELECT / FROM / WHERE / ORDER BY** — choose columns, from which table, filtered how, sorted how. Four keywords cover most day-to-day querying.
2. **JOIN** — combining tables on a shared key. **INNER JOIN** keeps only matching rows. **LEFT JOIN** keeps everything from the left table, filling gaps with NULL. This distinction is the most commonly tested SQL concept in BA interviews, because it changes your answer: an inner join silently drops customers with no orders.

3. **GROUP BY / HAVING** — collapsing rows into groups, then filtering those groups. WHERE filters rows before grouping; HAVING filters after.
4. **Aggregate functions** — COUNT, SUM, AVG, MIN, MAX. The arithmetic of reporting.
5. **DISTINCT** — removing duplicate rows from results.
6. **Alias** — a temporary short name for a table or column, keeping queries readable.
7. **Subquery / CTE** — a query inside a query. A CTE (the WITH clause) does the same job but reads top to bottom, which makes complex logic far easier to follow.
8. **UNION vs UNION ALL** — stacking result sets vertically. UNION removes duplicates and is slower; UNION ALL keeps everything.
9. **NULL / COALESCE** — NULL means unknown, not zero, and behaves oddly in comparisons. COALESCE substitutes a fallback value.
10. **CASE WHEN** — inline if-then logic, used constantly for bucketing values into categories.
11. **LIKE** — pattern matching with wildcards, e.g. finding all emails ending in a particular domain.
12. **Primary key / Foreign key / Index** — the unique identifier of a row; the field pointing at another table's key; a structure that speeds up lookups.
13. **View** — a saved query that behaves like a table. Useful when the same complex join is needed repeatedly.
14. **Stored procedure** — saved, reusable SQL that can take parameters.
15. **DDL vs DML** — DDL defines structure (CREATE, ALTER, DROP); DML manipulates data (SELECT, INSERT, UPDATE, DELETE). As a BA you'll almost always have read-only access, which means DML SELECT only.
16. **Window functions** — calculations across a set of rows without collapsing them, e.g. ranking each customer's orders by date.
ROW_NUMBER() OVER (PARTITION BY customer ORDER BY date). Advanced, but powerful.

❖ **Power BI and Tableau**

1. **Data source / Connector** — where the data comes from and the component that talks to it.

2. **Import vs DirectQuery** (Power BI) / **Extract vs Live** (Tableau) — either copy the data into the tool for speed, or query the source live for freshness. A real trade-off worth understanding.
3. **Data model / Relationships** — how tables link together inside the tool. Get this wrong and every number on the dashboard is wrong.
4. **Star schema / Fact table / Dimension table** — the standard reporting structure. The fact table holds the events and numbers (sales transactions); dimension tables hold the descriptive context (customer, product, date). Modelling this way makes dashboards fast and understandable.
5. **Cardinality** — whether a relationship is one-to-many or many-to-many. Many-to-many relationships are where double-counted totals come from.
6. **Measure vs Calculated column** — a measure is calculated on the fly as you slice the data; a calculated column is computed once per row and stored. Choosing wrongly causes both wrong numbers and slow reports.
7. **DAX** — Power BI's formula language for measures. **LOD expressions** are Tableau's rough equivalent.
8. **Filter / Slicer / Drill-through / Drill-down** — narrowing what's shown, either through controls or by clicking into detail. Drill-down goes deeper in the same visual; drill-through jumps to another page filtered to your selection.
9. **Dimension vs Measure** — a dimension is something you group by (region, month); a measure is something you count or sum (revenue, units).
10. **Row-level security (RLS)** — rules that show each user only their own slice of the data, so a regional manager sees only their region from the same report.
11. **Refresh schedule** — when the data automatically updates. Often the source of "why is this number stale" complaints.
12. **Workspace / Publish** — where reports live once shared, and the act of pushing them there.

❖ **Figma**

1. **Frame** — the container representing a screen or section. Everything lives inside frames.

2. **Component / Instance** — a reusable element defined once (a button) and reused everywhere. Change the component and every instance updates. This is how design systems stay consistent.
3. **Variant** — the states of a component grouped together: default, hover, disabled, error.
4. **Auto layout** — elements that resize and reflow automatically as content changes, rather than being pinned in place. Makes prototypes behave more like real screens.
5. **Design system / Style library** — the shared set of colours, fonts, spacing and components a product is built from.
6. **Prototype mode / Hotspot / Interaction** — linking screens so clicking an element moves to another screen. This is what turns a picture into something a stakeholder can actually try.
7. **Flow** — a named sequence of connected screens, e.g. the checkout flow.
8. **Comment mode** — leaving feedback pinned to a specific spot on a design. This is where a BA usually operates.
9. **Dev mode / Inspect** — where developers read exact spacing, colours and CSS from the design instead of guessing.
10. **Fidelity** — how finished it looks. Low-fidelity keeps feedback on structure and flow; high-fidelity invites comments about colours. Choose deliberately.

❖ **Lucidchart / Visio / draw.io**

1. **Shape library / Stencil** — the palette of symbols available, e.g. the BPMN set or the UML set. Loading the right one keeps your diagram standards-compliant.
2. **Canvas** — the drawing surface.
3. **Container** — a shape that holds other shapes and moves them together.
4. **Swimlane** — lanes assigning each step to a role or department, which is what exposes handoffs.
5. **Connector / Anchor point** — the arrows between shapes, and the fixed points they attach to so they stay connected when shapes move.
6. **Layer** — separating parts of a diagram so you can show or hide detail for different audiences.
7. **Standard flowchart symbols** — rectangle for a process step, diamond for a decision, rounded rectangle for start and end, cylinder for a data store. Using them correctly means readers don't need a legend.

8. **Export / Embed** — saving as PNG, SVG or PDF, or embedding the live diagram in Confluence so it updates in place rather than going out of date.

❖ **Miro / Mural**

1. **Board** — the infinite canvas everyone works on together.
2. **Sticky note** — the basic unit of workshop input, one idea per note.
3. **Frame** — a defined region of the board used to structure the session and to present.
4. **Timer** — a visible countdown. Sounds trivial, but timeboxing is the difference between a workshop and a two-hour meandering discussion.
5. **Dot voting** — participants spend a limited number of votes across options. Surfaces group priority quickly without the loudest person deciding.
6. **Affinity mapping / Clustering** — grouping scattered sticky notes into themes. The standard way to make sense of a brainstorm.
7. **Parking lot** — a frame for off-topic but valid points, so they're captured without derailing the session.
8. **Facilitation** — running the session: setting the goal, keeping time, drawing out quiet participants, closing with agreed actions. The actual skill; the tool is secondary.
9. **Common templates** — **empathy map** (what a user thinks, feels, says, does), **journey map** (their experience over time), **impact/effort matrix** (prioritisation on two axes), **retrospective board**.

❖ **Azure DevOps**

1. **Work item** — ADO's equivalent of a Jira issue: Epic, Feature, User Story, Task, Bug.
2. **Area path** — which team or product area owns the item. **Iteration path** — which sprint it belongs to. Together they replace Jira's project-and-sprint model.
3. **Boards / Repos / Pipelines / Test Plans / Artifacts** — the five hubs. Boards is planning, Repos is code, Pipelines is build and deploy, Test Plans is testing, Artifacts is packages. A BA lives mainly in Boards and Test Plans.
4. **Query / WIQL** — ADO's saved searches and its query language, equivalent to JQL.

5. **Backlog levels** — the hierarchy view letting you move between epic, feature and story level.
6. **Delivery plan** — a timeline view across multiple teams, useful for spotting dependency clashes.
7. **Pull request / Branch** — how code changes are proposed and reviewed. Worth recognising so you can follow developer conversations.

❖ **Test management (Xray, Zephyr, TestRail)**

1. **Test case** — the documented scenario being tested.
2. **Test step / Expected result / Actual result** — the instructions, what should happen, what did happen. The gap between the last two is a defect.
3. **Test suite / Test set** — a grouped collection of related cases.
4. **Test plan** — the overall strategy: what's in scope, who tests, in what environment, with what entry and exit criteria.
5. **Test execution / Test cycle** — an actual run of cases at a point in time. The same case runs many times across cycles.
6. **Pass / Fail / Blocked / Not executed** — result states. **Blocked** means it couldn't be tested, usually because something upstream is broken; distinguishing this from Fail matters for reporting.
7. **Precondition** — what must be true before the test starts, e.g. a customer account already exists.
8. **Test data** — the records needed to run the test. Regularly the thing that delays UAT.
9. **Coverage / Traceability report** — confirmation that every requirement has at least one test. This is where the RTM becomes real.
10. **Regression pack** — the standing set of tests rerun every release to confirm existing behaviour still works.
11. **Smoke pack** — a short set run immediately after deployment to confirm the system is broadly alive before deeper testing starts.

❖ **Postman**

1. **Request / Collection** — a single API call, and a saved group of them.
2. **Environment / Variable** — stored values like base URLs and tokens, so the same collection can run against dev, test and production by switching one dropdown.
3. **HTTP methods** — **GET** retrieves, **POST** creates, **PUT** replaces, **PATCH** updates part, **DELETE** removes.

4. **Endpoint** — the specific URL a request targets.
5. **Header / Body / Query parameter / Path parameter** — metadata sent with the request; the payload; filters appended after ?; values embedded in the URL path.
6. **JSON** — the standard data format for API payloads. Readable once you get used to the brackets, and worth being able to read.
7. **Status codes** — **200** success, **201** created, **400** bad request, **401** unauthenticated, **403** forbidden, **404** not found, **500** server error. The 400s mean the caller did something wrong; the 500s mean the server did.
8. **Authentication** — API key, Bearer token, OAuth. How the caller proves who it is.
9. **Mock server** — a fake API returning sample responses, letting front-end work start before the real API exists.
10. Worth knowing even if you never test APIs, because integration requirements get written in this language.